Severity Prediction

Purpose

Predict whether a COVID-19 VAERS adverse event report will be classified as **serious** (`SERIOUS = 1`) using structured clinical features and free-text symptom narratives.

Models compared

- Logistic Regression (with TF-IDF text + structured features)
- Decision Tree
- Random Forest

Data flow

1. Load pre-imputed VAERS dataset
2. Load precomputed comorbidity indicators
3. Clean symptom text — separate pipelines for supervised (leakage-scrubbed) and clustering tasks
4. Build feature matrix: numeric + categorical + manufacturer + comorbidity + TF-IDF text
5. Train models with stratified k-fold cross-validation and grid search
6. Compare structured-only vs structured+text feature sets
7. Extract and save feature importance for Logistic Regression, Random Forest, and Decision Tree

1. Setup & Imports

In [1]:
```
# 1. SETUP

import os
import re
import sys
import json
import math
import warnings
from collections import defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import Normalizer, OneHotEncoder
from sklearn.cluster import MiniBatchKMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_val
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    average_precision_score,
    f1_score,
    precision_score,
    recall_score,
    confusion_matrix
)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif

import scipy.sparse as sp

warnings.filterwarnings("ignore")

print("Python executable:", sys.executable)
print("Python version:", sys.version.split()[0])
print("Pandas version:", pd.__version__)
```

```
Python executable: /Users/ariellerothman/Desktop/Sperm Cell Analysis/.venv/b
in/python
Python version: 3.12.10
Pandas version: 2.2.3
```

2. Load Data

Load the preprocessed VAERS dataset produced by `Data preparation.ipynb` (expected at `Outputs/df_clean_imputed.csv` or `Outputs/df_clean_engineered.csv`). If not found, re-run that notebook first.

In [2]:
```
# 2. LOAD PREPROCESSED DATASET
from pathlib import Path

HERE = Path().resolve()  # project root (the folder containing this notebook
candidate_paths = [
    str(HERE / "Outputs" / "df_clean_imputed.csv"),
    str(HERE / "Outputs" / "df_clean_engineered.csv"),
]

data_path = next((p for p in candidate_paths if os.path.exists(p)), None)

if data_path is not None:
    df_clean = pd.read_csv(data_path, low_memory=False)
    print(f"✓ Loaded dataset from: {data_path}")
    print(f"  Shape: {df_clean.shape}")
else:
```

```
    raise FileNotFoundError(
        "Expected one of these files but none were found:\n"
        + "\n".join(candidate_paths)
    )
```

```
✓ Loaded dataset from: /Users/ariellerothman/Desktop/Capstone/Outputs/df_cle
an_imputed.csv
  Shape: (986096, 113)
```

3. Load Precomputed Comorbidity Indicators

In [3]:

```
# ============================================================
# 3. LOAD PRECOMPUTED COMORBIDITY INDICATORS
# ============================================================

import os
import pandas as pd

from pathlib import Path
HERE   = Path().resolve()  # project root
OUTDIR = str(HERE / "Outputs")
os.makedirs(OUTDIR, exist_ok=True)

COMORB_PATH = str(HERE / "Outputs" / "comorbidity_indicators.csv")

# Detect currently available indicator columns in df_clean
existing_indicator_cols = [
    c for c in df_clean.columns
    if "__" in c or c.endswith("_MISSING")
]

print(f"Existing indicator columns already in df_clean: {len(existing_indica

if not os.path.exists(COMORB_PATH):
    raise FileNotFoundError(
        f"Comorbidity indicator file not found: {COMORB_PATH}\n"
        "Run Data preparation notebook first to generate comorbidity_indicat
    )

comorb_df = pd.read_csv(COMORB_PATH, low_memory=False)

if "VAERS_ID" not in comorb_df.columns:
    raise ValueError("comorbidity_indicators.csv must contain VAERS_ID for m

indicator_cols = [
    c for c in comorb_df.columns
    if c != "VAERS_ID" and ("__" in c or c.endswith("_MISSING"))
]

missing_in_df = [c for c in indicator_cols if c not in df_clean.columns]

if not missing_in_df:
    print("All comorbidity indicator columns are already present. Skipping m
else:
    merge_cols = ["VAERS_ID"] + missing_in_df
```

```
    # Use one row per VAERS_ID to avoid accidental row multiplication
    comorb_merge = comorb_df[merge_cols].drop_duplicates(subset=["VAERS_ID"]

    df_clean = df_clean.merge(
        comorb_merge,
        on="VAERS_ID",
        how="left",
        validate="m:1"
    )

    # Fill unmatched IDs as 0 and cast to compact integer dtype
    for c in missing_in_df:
        df_clean[c] = df_clean[c].fillna(0).astype("int8")

    print(f"Merged {len(missing_in_df)} indicator columns from comorbidity_i

final_indicator_cols = [
    c for c in df_clean.columns
    if "__" in c or c.endswith("_MISSING")
]

print(f"Total indicator columns now in df_clean: {len(final_indicator_cols)}

print("Sample indicator columns:")
print(final_indicator_cols[:20])
```

```
Existing indicator columns already in df_clean: 64
All comorbidity indicator columns are already present. Skipping merge.
Total indicator columns now in df_clean: 64
Sample indicator columns:
['MANU__JANSSEN', 'MANU__MODERNA', 'MANU__NOVAVAX', 'MANU__PFIZER_BIONTECH',
'MANU__UNKNOWN_MANUFACTURER', 'AGE_YRS_MISSING', 'HOSPDAYS_MISSING', 'NUMDAY
S_MISSING', 'ONSET_INTERVAL_MISSING', 'MAX_DOSE_MISSING', 'HISTORY_MISSING',
'HISTORY__cardiovascular', 'HISTORY__respiratory', 'HISTORY__metabolic', 'HI
STORY__endocrine_thyroid', 'HISTORY__musculoskeletal', 'HISTORY__mental_heal
th', 'HISTORY__gastrointestinal', 'HISTORY__kidney', 'HISTORY__clotting']
```

4. Symptom Text Preprocessing

Column	Purpose
`SYMPTOM_TEXT_CLEAN`	Base cleaning (lowercased, URLs/numbers removed)
`SYMPTOM_TEXT_CLEAN_NOLEAK`	Supervised modelling — leakage terms scrubbed (hospitalization, death, outcome labels)

In [4]:
```
# ============================================================
# 4. SYMPTOM TEXT PREPROCESSING
# ============================================================
# Produces two text columns for downstream use:
# - SYMPTOM_TEXT_CLEAN: base cleaning only
# - SYMPTOM_TEXT_CLEAN_NOLEAK: leakage-scrubbed for supervised modelling

import re
```

```python
import pandas as pd

# ----------------------------------------------------
# Base text cleaning
# ----------------------------------------------------
_url_re = re.compile(r"http\S+|www\S+|https\S+")
_email_re = re.compile(r"\S+@\S+")
_num_re = re.compile(r"\d+")
_keep_re = re.compile(r"[^a-z0-9\s\-]")
_ws2 = re.compile(r"\s+")

def clean_symptom_text(x) -> str:
    """
    Base-level cleaning for symptom text before any model-specific scrubbing

    Steps:
        1. Lowercase and strip whitespace
        2. Remove URLs and email addresses
        3. Replace numeric tokens with placeholder '__NUM__'
        4. Remove non-alphanumeric characters (except hyphens)
        5. Collapse multiple whitespace

    Args:
        x: Raw symptom text string (or NaN).

    Returns:
        Cleaned lowercase string, or empty string if input is null.
    """
    if pd.isna(x) or x == "":
        return ""
    x = str(x).strip().lower()
    x = _url_re.sub(" ", x)
    x = _email_re.sub(" ", x)
    x = _num_re.sub(" __NUM__ ", x)
    x = _keep_re.sub(" ", x)
    x = _ws2.sub(" ", x).strip()
    return x

df_clean["SYMPTOM_TEXT_CLEAN"] = df_clean["SYMPTOM_TEXT"].apply(clean_symptc

# ----------------------------------------------------
# Strong supervised leakage scrubbing for severity prediction
# Removes direct outcome / adjudication / downstream-care terms
# plus residual administrative/report-style phrases.
#
# WHY: The VAERS SERIOUS label is defined by hospitalization, death,
# disability, and life-threatening events. If those words appear
# verbatim in the symptom text, any classifier trivially learns to
# predict the label from them rather than real clinical patterns.
# ----------------------------------------------------
LEAKAGE_PATTERNS = [re.compile(p) for p in [

    # ----------------------------------------------
    # Hospital / admission / downstream care
    # ----------------------------------------------
    r"\bhospital\w*\b",
```

```
    r"\badmit\w*\b",
    r"\binpatient\b",
    r"\bdischarg\w*\b",
    r"\bed\b",
    r"\ber\b",
    r"\bemergency\b",
    r"\bemergency room\b",
    r"\burgent\b",
    r"\burgent care\b",
    r"\btransport\w*\b",
    r"\btransfer\w*\b",
    r"\bambulance\b",
    r"\bems\b",
    r"\bhospice\b",

    # ------------------------------------------------
    # Death / life-threatening / rescue care
    # ------------------------------------------------
    r"\bdeath\b",
    r"\bdied\b",
    r"\bdead\b",
    r"\bfatal\b",
    r"\bpassed away\b",
    r"\bautopsy\b",
    r"\blife[-\s]?threaten\w*\b",
    r"\bintubat\w*\b",
    r"\bventilat\w*\b",
    r"\bicu\b",
    r"\barrest\b",

    # ------------------------------------------------
    # Seriousness / adjudication labels
    # ------------------------------------------------
    r"\bserious\w*\b",
    r"\bnon[-\s]?serious\w*\b",
    r"\bmedically significant\b",
    r"\bmedically important\b",
    r"\bcriterion\b",
    r"\bcriteria\b",
    r"\boutcome\b",
    r"\bdisability\b",
    r"\bpermanent\b",
    r"\bprolonged\b",

    # ------------------------------------------------
    # Administrative / report-template phrasing
    # ------------------------------------------------
    r"\breported cause\b",
    r"\breport medically\b",
    r"\bcriterion hospitalization\b",
    r"\bhospitalization medically\b",
    r"\bsignificant outcome\b",
    r"\bnarrative\b",
    r"\breport\b",
    r"\breported\b",
    r"\bspontaneous\b",
```

```
    r"\bcase\b",
    r"\bsource\b",
    r"\bpatient unknown\b",
    r"\bunknown\b",

    # ----------------------------------------------
    # Residual report-style / quasi-administrative phrases
    # seen in current model outputs
    # ----------------------------------------------
    r"\bevent resulted\b",
    r"\bevents resulted\b",
    r"\bresulted\b",
    r"\bcaused prolonged\b",
    r"\bmedical center\b",
    r"\bclinic care\b",
    r"\bwent care\b",
    r"\bvisited\b",
    r"\bperformed\b",

    # ----------------------------------------------
    # Other report-level phrasing seen previously
    # ----------------------------------------------
    r"\broom\b",
    r"\bdepartment\b",
    r"\bvisit\b",
    r"\bwent room\b",
    r"\bwent doctor\b",
    r"\bdoctor visit\b"
]]

def scrub_leakage_terms_strong(x) -> str:
    """
    Remove supervised leakage terms from cleaned symptom text.

    Args:
        x: Cleaned symptom text string (output of clean_symptom_text).

    Returns:
        Text with leakage terms replaced by spaces and whitespace collapsed.
    """
    if pd.isna(x) or x == "":
        return ""
    x = str(x)
    for p in LEAKAGE_PATTERNS:
        x = p.sub(" ", x)
    x = _ws2.sub(" ", x).strip()
    return x

df_clean["SYMPTOM_TEXT_CLEAN_NOLEAK"] = (
    df_clean["SYMPTOM_TEXT_CLEAN"]
    .apply(scrub_leakage_terms_strong)
)

print("Text columns created:")
print([c for c in df_clean.columns if "SYMPTOM" in c])
```

```
Text columns created:
['SYMPTOM_TEXT', 'SYMPTOM_TEXT_CHAR_LEN', 'SYMPTOM_TEXT_CLEAN', 'SYMPTOM_TEX
T_CLEAN_NOLEAK']
```

5. Supervised Feature Matrix

Define which columns from `df_clean` enter the model as features.

In [5]:
```
# 5. SUPERVISED FEATURE MATRIX SETUP
#     Cleaned final version for severity prediction

TEXT_FEATURE = "SYMPTOM_TEXT_CLEAN_NOLEAK"
y = df_clean["SERIOUS"].astype(int)

# Numeric features
# Keep onset-timing variables for the main model
# Remove HOSPDAYS because it is downstream / leakage-prone
numeric_cols = [
    c for c in ["AGE_YRS", "NUMDAYS", "ONSET_INTERVAL", "MAX_DOSE"]
    if c in df_clean.columns
]

# Coerce selected numeric columns to numeric dtype.
# This handles string artifacts like "UNKNOWN" from upstream exports.
for c in numeric_cols:
    df_clean[c] = pd.to_numeric(df_clean[c], errors="coerce")

# Categorical features
categorical_cols = [
    c for c in ["SEX", "STATE", "YEAR", "MONTH"]
    if c in df_clean.columns
]

# Ensure categoricals are treated consistently as strings for the categorica
for c in categorical_cols:
    df_clean[c] = df_clean[c].astype("string")

# Manufacturer and dose-related features
manufacturer_cols = [c for c in df_clean.columns if c.startswith("MANU__")]

dose_cols = [
    c for c in ["DOSE_COUNT", "MULTI_DOSE", "UNKNOWN_DOSE", "MULTI_MANUFACTU
    if c in df_clean.columns
]

# Missingness indicators
missing_cols = [
    c for c in [
        "NUMDAYS_MISSING",
        "ONSET_INTERVAL_MISSING",
        "AGE_YRS_MISSING",
        "HISTORY_MISSING",
        "CUR_ILL_MISSING",
        "ALLERGIES_MISSING",
```

```
        "PRIOR_VAX_MISSING",
        "LAB_DATA_MISSING",
        "OTHER_MEDS_MISSING"
    ]
    if c in df_clean.columns
]

# Engineered clinical / comorbidity binary features
comorb_prefixes = (
    "HISTORY__",
    "CUR_ILL__",
    "ALLERGIES__",
    "PRIOR_VAX__",
    "LAB_DATA__",
    "OTHER_MEDS__"
)

comorb_cols = [c for c in df_clean.columns if c.startswith(comorb_prefixes)]

# Explicit exclusions for leakage-prone / helper / raw columns
exclude_cols = {
    # target / direct leakage
    "SERIOUS",
    "DIED",
    "L_THREAT",
    "ER_VISIT",
    "ER_ED_VISIT",
    "HOSPITAL",
    "DISABLE",
    "BIRTH_DEFECT",
    "HOSPDAYS",
    "DATEDIED",

    # identifiers / raw dates
    "VAERS_ID",
    "RECVDATE",
    "VAX_DATE",
    "ONSET_DATE",

    # raw free-text fields (already engineered elsewhere)
    "SYMPTOM_TEXT",
    "LAB_DATA",
    "OTHER_MEDS",
    "CUR_ILL",
    "HISTORY",
    "PRIOR_VAX",
    "ALLERGIES",

    # normalized helper text columns
    "HISTORY_NORM",
    "CUR_ILL_NORM",
    "ALLERGIES_NORM",
    "PRIOR_VAX_NORM",
    "LAB_DATA_NORM",
    "OTHER_MEDS_NORM",
```

```python
    # text/clustering helper columns not used as structured inputs
    "SYMPTOM_TEXT_CLEAN",
    "SYMPTOM_TEXT_CLEAN_CLUSTER",
    "CLUSTER_TOKEN_COUNT",

    # optional: usually unhelpful if dataset is already one vaccine type
    "VAX_TYPE"
}

# Final structured feature list
structured_cols = (
    numeric_cols +
    categorical_cols +
    manufacturer_cols +
    dose_cols +
    missing_cols +
    comorb_cols
)

# Remove anything explicitly excluded
structured_cols = [c for c in structured_cols if c not in exclude_cols]

# Remove duplicates while preserving order
structured_cols = list(dict.fromkeys(structured_cols))

# Diagnostics
print("TEXT_FEATURE:", TEXT_FEATURE)
print("Target:", "SERIOUS")
print()

print("Numeric cols:", numeric_cols)
print("Categorical cols:", categorical_cols)
print("Manufacturer cols:", manufacturer_cols)
print("Dose cols:", dose_cols)
print("Missing cols:", missing_cols)
print("Comorbidity/clinical cols count:", len(comorb_cols))
print("Total structured feature count:", len(structured_cols))
print()

if numeric_cols:
    print("Numeric dtypes after coercion:")
    print(df_clean[numeric_cols].dtypes)
```

```
TEXT_FEATURE: SYMPTOM_TEXT_CLEAN_NOLEAK
Target: SERIOUS

Numeric cols: ['AGE_YRS', 'NUMDAYS', 'ONSET_INTERVAL', 'MAX_DOSE']
Categorical cols: ['SEX', 'STATE', 'YEAR', 'MONTH']
Manufacturer cols: ['MANU__JANSSEN', 'MANU__MODERNA', 'MANU__NOVAVAX', 'MANU
__PFIZER_BIONTECH', 'MANU__UNKNOWN_MANUFACTURER']
Dose cols: ['DOSE_COUNT', 'MULTI_DOSE', 'UNKNOWN_DOSE', 'MULTI_MANUFACTURE
R']
Missing cols: ['NUMDAYS_MISSING', 'ONSET_INTERVAL_MISSING', 'AGE_YRS_MISSIN
G', 'HISTORY_MISSING', 'CUR_ILL_MISSING', 'ALLERGIES_MISSING', 'PRIOR_VAX_MI
SSING', 'LAB_DATA_MISSING', 'OTHER_MEDS_MISSING']
Comorbidity/clinical cols count: 48
Total structured feature count: 74

Numeric dtypes after coercion:
AGE_YRS           float64
NUMDAYS           float64
ONSET_INTERVAL    float64
MAX_DOSE          float64
dtype: object
```

6. Column Inspection (Diagnostic)

Displays all columns in `df_clean` with their dtypes. Use this to verify that comorbidity indicators, text variants, and other engineered columns were created correctly before building the feature matrix.

In [6]:

```python
# 6. COLUMN INSPECTION (DIAGNOSTIC)
import pandas as pd

print("Total columns:", len(df_clean.columns))

pd.set_option("display.max_rows", None)

col_df = pd.DataFrame({
    "column": df_clean.columns,
    "dtype": df_clean.dtypes
})

display(col_df)
```

```
Total columns: 115
```

	column	dty
VAERS_ID	VAERS_ID	in
DIED	DIED	in
L_THREAT	L_THREAT	in
ER_VISIT	ER_VISIT	in
ER_ED_VISIT	ER_ED_VISIT	in
HOSPITAL	HOSPITAL	in
DISABLE	DISABLE	in
BIRTH_DEFECT	BIRTH_DEFECT	in
RECVDATE	RECVDATE	obj
VAX_DATE	VAX_DATE	obj
ONSET_DATE	ONSET_DATE	obj
DATEDIED	DATEDIED	obj
AGE_YRS	AGE_YRS	floa
HOSPDAYS	HOSPDAYS	floa
NUMDAYS	NUMDAYS	floa
SEX	SEX	string[pyth
STATE	STATE	string[pyth
VAX_TYPE	VAX_TYPE	obj
SYMPTOM_TEXT	SYMPTOM_TEXT	obj
LAB_DATA	LAB_DATA	obj
OTHER_MEDS	OTHER_MEDS	obj
CUR_ILL	CUR_ILL	obj
HISTORY	HISTORY	obj
PRIOR_VAX	PRIOR_VAX	obj
ALLERGIES	ALLERGIES	obj
VAX_MANU	VAX_MANU	obj
VAX_NAME	VAX_NAME	obj
VAX_DOSE_SERIES	VAX_DOSE_SERIES	obj
VAX_ROUTE	VAX_ROUTE	obj
VAX_SITE	VAX_SITE	obj
VAX_LOT	VAX_LOT	obj
SERIOUS	SERIOUS	in

	column	dty
SEVERE	SEVERE	in
ONSET_INTERVAL	ONSET_INTERVAL	floa
YEAR	YEAR	string[pyth
MONTH	MONTH	string[pyth
MAX_DOSE	MAX_DOSE	floa
DOSE_COUNT	DOSE_COUNT	in
MULTI_DOSE	MULTI_DOSE	in
UNKNOWN_DOSE	UNKNOWN_DOSE	in
MANU__JANSSEN	MANU__JANSSEN	in
MANU__MODERNA	MANU__MODERNA	in
MANU__NOVAVAX	MANU__NOVAVAX	in
MANU__PFIZER_BIONTECH	MANU__PFIZER_BIONTECH	in
MANU__UNKNOWN_MANUFACTURER	MANU__UNKNOWN_MANUFACTURER	in
MULTI_MANUFACTURER	MULTI_MANUFACTURER	in
AGE_YRS_MISSING	AGE_YRS_MISSING	in
HOSPDAYS_MISSING	HOSPDAYS_MISSING	in
NUMDAYS_MISSING	NUMDAYS_MISSING	in
ONSET_INTERVAL_MISSING	ONSET_INTERVAL_MISSING	in
MAX_DOSE_MISSING	MAX_DOSE_MISSING	in
SYMPTOM_TEXT_CHAR_LEN	SYMPTOM_TEXT_CHAR_LEN	in
YEAR_MONTH	YEAR_MONTH	obj
HISTORY_NORM	HISTORY_NORM	obj
HISTORY_MISSING	HISTORY_MISSING	in
HISTORY__cardiovascular	HISTORY__cardiovascular	in
HISTORY__respiratory	HISTORY__respiratory	in
HISTORY__metabolic	HISTORY__metabolic	in
HISTORY__endocrine_thyroid	HISTORY__endocrine_thyroid	in
HISTORY__musculoskeletal	HISTORY__musculoskeletal	in
HISTORY__mental_health	HISTORY__mental_health	in
HISTORY__gastrointestinal	HISTORY__gastrointestinal	in
HISTORY__kidney	HISTORY__kidney	in
HISTORY__clotting	HISTORY__clotting	in

	column	dt
HISTORY__autoimmune	HISTORY__autoimmune	in
HISTORY__cancer_immunocompromised	HISTORY__cancer_immunocompromised	in
HISTORY__neurologic	HISTORY__neurologic	in
HISTORY__lymphatic_edema	HISTORY__lymphatic_edema	in
HISTORY__other	HISTORY__other	in
CUR_ILL_NORM	CUR_ILL_NORM	ob
CUR_ILL_MISSING	CUR_ILL_MISSING	in
CUR_ILL__acute_infection	CUR_ILL__acute_infection	in
CUR_ILL__chronic_cardiovascular	CUR_ILL__chronic_cardiovascular	in
CUR_ILL__chronic_respiratory	CUR_ILL__chronic_respiratory	in
CUR_ILL__metabolic_endocrine	CUR_ILL__metabolic_endocrine	in
CUR_ILL__endocrine_thyroid	CUR_ILL__endocrine_thyroid	in
CUR_ILL__autoimmune_inflammatory	CUR_ILL__autoimmune_inflammatory	in
CUR_ILL__neurologic	CUR_ILL__neurologic	in
CUR_ILL__neuro_headache	CUR_ILL__neuro_headache	in
CUR_ILL__cancer_immunocompromised	CUR_ILL__cancer_immunocompromised	in
CUR_ILL__lymphatic_edema	CUR_ILL__lymphatic_edema	in
CUR_ILL__other	CUR_ILL__other	in
ALLERGIES_NORM	ALLERGIES_NORM	ob
ALLERGIES_MISSING	ALLERGIES_MISSING	in
ALLERGIES__drug_allergy	ALLERGIES__drug_allergy	in
ALLERGIES__food_allergy	ALLERGIES__food_allergy	in
ALLERGIES__latex_other_contact	ALLERGIES__latex_other_contact	in
ALLERGIES__anaphylaxis_history	ALLERGIES__anaphylaxis_history	in
ALLERGIES__other	ALLERGIES__other	in
PRIOR_VAX_NORM	PRIOR_VAX_NORM	ob
PRIOR_VAX_MISSING	PRIOR_VAX_MISSING	in
PRIOR_VAX__prior_covid_vax	PRIOR_VAX__prior_covid_vax	in
PRIOR_VAX__prior_flu_vax	PRIOR_VAX__prior_flu_vax	in
PRIOR_VAX__prior_other_vax	PRIOR_VAX__prior_other_vax	in
PRIOR_VAX__prior_vax_reaction	PRIOR_VAX__prior_vax_reaction	in
PRIOR_VAX__other	PRIOR_VAX__other	in

	column	dty
LAB_DATA_NORM	LAB_DATA_NORM	obj
LAB_DATA_MISSING	LAB_DATA_MISSING	in
LAB_DATA__cardiac_markers	LAB_DATA__cardiac_markers	in
LAB_DATA__coagulation	LAB_DATA__coagulation	in
LAB_DATA__inflammation	LAB_DATA__inflammation	in
LAB_DATA__cbc	LAB_DATA__cbc	in
LAB_DATA__vitals_procedures	LAB_DATA__vitals_procedures	in
LAB_DATA__other	LAB_DATA__other	in
OTHER_MEDS_NORM	OTHER_MEDS_NORM	obj
OTHER_MEDS_MISSING	OTHER_MEDS_MISSING	in
OTHER_MEDS__anticoagulant	OTHER_MEDS__anticoagulant	in
OTHER_MEDS__statin	OTHER_MEDS__statin	in
OTHER_MEDS__immunosuppressant	OTHER_MEDS__immunosuppressant	in
OTHER_MEDS__diabetes_meds	OTHER_MEDS__diabetes_meds	in
OTHER_MEDS__thyroid_meds	OTHER_MEDS__thyroid_meds	in
OTHER_MEDS__psych_meds	OTHER_MEDS__psych_meds	in
OTHER_MEDS__other	OTHER_MEDS__other	in
SYMPTOM_TEXT_CLEAN	SYMPTOM_TEXT_CLEAN	obj
SYMPTOM_TEXT_CLEAN_NOLEAK	SYMPTOM_TEXT_CLEAN_NOLEAK	obj

7. Model Training: CV-Based Comparative Analysis + Final Held-Out Test

A stratified working sample of 200,000 rows is drawn from the full dataset to keep runtimes tractable while preserving the class distribution (~19.85% serious).

For each model, we use a two-stage design to separate model selection from model comparison and final confirmation:

1. **Draw a stratified 200k working sample** from the full dataset
2. **Hold out 20%** (40k rows) as a final test set and keep it locked until the end
3. Use a **20k stratified tuning subset** for hyperparameter selection via stratified 3-fold CV
4. Report **mean ± SD** across folds for PR-AUC, F1, precision, and recall using the full 160k training set (CV experimental design)

5. Refit each selected model on the full 160k training set and evaluate once on the held-out 40k test set

In [8]:
```
# 7. MODEL TRAINING: CV-BASED COMPARISON + FINAL HELD-OUT TEST

from sklearn.model_selection import StratifiedKFold, GridSearchCV, train_tes
from sklearn.metrics import average_precision_score, f1_score, precision_sco
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, MaxAbsScaler
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import numpy as np

df_model = df_clean.copy()
active_text_feature = TEXT_FEATURE

# ---------------------------------------------------
# 0. Stratified 200k working sample
# ---------------------------------------------------
WORKING_SAMPLE_SIZE = 200_000

df_work, _ = train_test_split(
    df_model,
    train_size=WORKING_SAMPLE_SIZE,
    stratify=df_model["SERIOUS"],
    random_state=42
)
df_model = df_work

print(f"Working sample: {len(df_model):,} rows  |  serious rate: {df_model['

# Derive active column lists from what actually exists in df_model
active_numeric_cols = [c for c in numeric_cols if c in df_model.columns]
active_categorical_cols = [c for c in categorical_cols if c in df_model.colu
active_structured_cols = [c for c in structured_cols if c in df_model.columr
active_binary_cols = [
    c for c in active_structured_cols
    if c not in (active_numeric_cols + active_categorical_cols)
]

# Build preprocessing pipeline
numeric_pipe_runtime = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_pipe_runtime = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

```
structured_preprocess_runtime = ColumnTransformer(
    transformers=[
        ("num", numeric_pipe_runtime, active_numeric_cols),
        ("cat", categorical_pipe_runtime, active_categorical_cols),
        ("binary", "passthrough", active_binary_cols),
    ],
    remainder="drop",
    sparse_threshold=1.0
)

text_preprocess_runtime = Pipeline([
    ("tfidf", TfidfVectorizer(
        max_features=5000,
        ngram_range=(1, 2),
        min_df=10,
        max_df=0.9,
        stop_words="english",
        sublinear_tf=True,
        norm="l2",
        strip_accents="unicode",
        token_pattern=r"(?u)\b[a-zA-Z][a-zA-Z\-]+\b|__NUM__"
    ))
])

full_preprocess_runtime = ColumnTransformer(
    transformers=[
        ("structured", structured_preprocess_runtime, active_structured_cols
        ("text", text_preprocess_runtime, active_text_feature),
    ],
    remainder="drop",
    sparse_threshold=1.0
)

print("Runtime feature setup:")
print("  Numeric:", len(active_numeric_cols))
print("  Categorical:", len(active_categorical_cols))
print("  Binary:", len(active_binary_cols))
print("  Structured total:", len(active_structured_cols))
print("  Text feature:", active_text_feature)

# ---------------------------------------------------
# 1. Train / test split on working sample (80 / 20)
# ---------------------------------------------------
df_train, df_test = train_test_split(
    df_model,
    test_size=0.20,
    stratify=df_model["SERIOUS"],
    random_state=42
)

y_train = df_train["SERIOUS"].astype(int)
y_test  = df_test["SERIOUS"].astype(int)

print(f"\nTrain set : {df_train.shape}  |  serious rate: {y_train.mean():.3f
print(f"Test  set : {df_test.shape}   |  serious rate: {y_test.mean():.3f}")
```

```python
# ----------------------------------------------------
# 1b. Tuning subset for GridSearchCV (~20k rows, ~12.5% of training set)
#     Keeps hyperparameter search fast; CV eval uses the full training set.
# ----------------------------------------------------
TUNE_SIZE = 20_000

df_tune, _ = train_test_split(
    df_train,
    train_size=TUNE_SIZE,
    stratify=y_train,
    random_state=42
)
y_tune = df_tune["SERIOUS"].astype(int)

print(f"Tuning subset : {df_tune.shape}  |  serious rate: {y_tune.mean():.3f

# ----------------------------------------------------
# 2. Models and parameter grids
# ----------------------------------------------------
CV_SPLITS = 3

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        solver="saga",
        n_jobs=-1,
        random_state=42
    ),
    "Decision Tree": DecisionTreeClassifier(
        class_weight="balanced",
        random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    )
}

light_param_grids = {
    "Logistic Regression": {
        "clf__C": [0.1, 1.0]
    },
    "Decision Tree": {
        "clf__max_depth": [10, 20],
        "clf__min_samples_split": [5, 10]
    },
    "Random Forest": {
        "clf__n_estimators": [100],
        "clf__max_depth": [10, 20]
    }
}

cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=42)
```

```python
cv_scoring = {
    "average_precision": "average_precision",
    "f1": "f1",
    "precision": "precision",
    "recall": "recall"
}

results = []
best_models = {}

print("Models to run:", list(models.keys()))

# ---------------------------------------------------
# 3. Tune, CV evaluate, then confirm on held-out test
# ---------------------------------------------------
for name, clf in models.items():
    print("\n" + "=" * 80)
    print("MODEL:", name)
    print("=" * 80)

    base_pipe = Pipeline([
        ("preprocess", full_preprocess_runtime),
        ("scale", MaxAbsScaler()),
        ("clf", clf)
    ])

    grid = GridSearchCV(
        estimator=base_pipe,
        param_grid=light_param_grids[name],
        scoring="average_precision",
        cv=cv,
        n_jobs=1,
        verbose=1,
        refit=True,
        error_score="raise"
    )

    # Stage A: Hyperparameter selection on tuning subset
    grid.fit(df_tune, y_tune)
    best_params = grid.best_params_
    print("Best params:", best_params)

    # Stage B: CV-based comparative evaluation with fixed params on full tra
    fixed_pipe = Pipeline([
        ("preprocess", full_preprocess_runtime),
        ("scale", MaxAbsScaler()),
        ("clf", clf)
    ])
    fixed_pipe.set_params(**best_params)

    cv_scores = cross_validate(
        estimator=fixed_pipe,
        X=df_train,
        y=y_train,
        scoring=cv_scoring,
        cv=cv,
```

```python
        n_jobs=1,
        return_train_score=False,
        error_score="raise"
    )

    cv_pr_auc_mean = cv_scores["test_average_precision"].mean()
    cv_pr_auc_std = cv_scores["test_average_precision"].std()
    cv_f1_mean = cv_scores["test_f1"].mean()
    cv_f1_std = cv_scores["test_f1"].std()
    cv_precision_mean = cv_scores["test_precision"].mean()
    cv_precision_std = cv_scores["test_precision"].std()
    cv_recall_mean = cv_scores["test_recall"].mean()
    cv_recall_std = cv_scores["test_recall"].std()

    print(f"CV PR-AUC:    {cv_pr_auc_mean:.4f} ± {cv_pr_auc_std:.4f}")
    print(f"CV F1:        {cv_f1_mean:.4f} ± {cv_f1_std:.4f}")
    print(f"CV Precision: {cv_precision_mean:.4f} ± {cv_precision_std:.4f}")
    print(f"CV Recall:    {cv_recall_mean:.4f} ± {cv_recall_std:.4f}")

    # Stage C: Final refit on full training set and one-time held-out test
    fixed_pipe.fit(df_train, y_train)
    best_models[name] = fixed_pipe

    y_pred_test = fixed_pipe.predict(df_test)
    y_prob_test = fixed_pipe.predict_proba(df_test)[:, 1]

    test_pr_auc = average_precision_score(y_test, y_prob_test)
    test_f1 = f1_score(y_test, y_pred_test, zero_division=0)
    test_precision = precision_score(y_test, y_pred_test, zero_division=0)
    test_recall = recall_score(y_test, y_pred_test, zero_division=0)
    cm = confusion_matrix(y_test, y_pred_test)

    print(f"Test PR-AUC:    {test_pr_auc:.4f}")
    print(f"Test F1:        {test_f1:.4f}")
    print(f"Test Precision: {test_precision:.4f}")
    print(f"Test Recall:    {test_recall:.4f}")
    print("Test Confusion Matrix:")
    print(cm)

    results.append({
        "Model": name,
        "BestParams": best_params,
        "CV_PR_AUC_Mean": cv_pr_auc_mean,
        "CV_PR_AUC_SD": cv_pr_auc_std,
        "CV_F1_Mean": cv_f1_mean,
        "CV_F1_SD": cv_f1_std,
        "CV_Precision_Mean": cv_precision_mean,
        "CV_Precision_SD": cv_precision_std,
        "CV_Recall_Mean": cv_recall_mean,
        "CV_Recall_SD": cv_recall_std,
        "Test_PR_AUC": test_pr_auc,
        "Test_F1": test_f1,
        "Test_Precision": test_precision,
        "Test_Recall": test_recall,
        "ConfusionMatrix": cm
    })
```

```
summary_df = pd.DataFrame([
    {k: v for k, v in r.items() if k != "ConfusionMatrix"}
    for r in results
])

print("\nMODEL COMPARISON - CV experimental design (ranked by CV PR-AUC)")
print(summary_df.sort_values("CV_PR_AUC_Mean", ascending=False))

summary_df.to_csv(f"{OUTDIR}/supervised_model_comparison_raw_tfidf.csv", inc
print("\nStored best models:", list(best_models.keys()))
```

```
Working sample: 200,000 rows  |  serious rate: 0.199
Runtime feature setup:
  Numeric: 4
  Categorical: 4
  Binary: 66
  Structured total: 74
  Text feature: SYMPTOM_TEXT_CLEAN_NOLEAK

Train set : (160000, 115)  |  serious rate: 0.199
Test  set : (40000, 115)   |  serious rate: 0.199
Tuning subset : (20000, 115)  |  serious rate: 0.199
Models to run: ['Logistic Regression', 'Decision Tree', 'Random Forest']

================================================================================
====
MODEL: Logistic Regression
================================================================================
====
Fitting 3 folds for each of 2 candidates, totalling 6 fits
Best params: {'clf__C': 1.0}
CV PR-AUC:    0.8013 ± 0.0008
CV F1:        0.7025 ± 0.0012
CV Precision: 0.6090 ± 0.0029
CV Recall:    0.8299 ± 0.0021
Test PR-AUC:    0.8038
Test F1:        0.7054
Test Precision: 0.6105
Test Recall:    0.8353
Test Confusion Matrix:
[[27828  4232]
 [ 1308  6632]]

================================================================================
====
MODEL: Decision Tree
================================================================================
====
Fitting 3 folds for each of 4 candidates, totalling 12 fits
Best params: {'clf__max_depth': 10, 'clf__min_samples_split': 10}
CV PR-AUC:    0.6322 ± 0.0045
CV F1:        0.6130 ± 0.0009
CV Precision: 0.5148 ± 0.0020
CV Recall:    0.7575 ± 0.0047
Test PR-AUC:    0.6435
Test F1:        0.6216
Test Precision: 0.5298
Test Recall:    0.7520
Test Confusion Matrix:
[[26760  5300]
 [ 1969  5971]]

================================================================================
====
MODEL: Random Forest
================================================================================
====
```

```
Fitting 3 folds for each of 2 candidates, totalling 6 fits
Best params: {'clf__max_depth': 20, 'clf__n_estimators': 100}
CV PR-AUC:    0.7382 ± 0.0023
CV F1:        0.6456 ± 0.0038
CV Precision: 0.5547 ± 0.0036
CV Recall:    0.7722 ± 0.0040
Test PR-AUC:    0.7396
Test F1:        0.6460
Test Precision: 0.5461
Test Recall:    0.7907
Test Confusion Matrix:
[[26842  5218]
 [ 1662  6278]]

MODEL COMPARISON - CV experimental design (ranked by CV PR-AUC)
                 Model                                        BestParams  C
V_PR_AUC_Mean  CV_PR_AUC_SD  CV_F1_Mean  CV_F1_SD  CV_Precision_Mean  CV_Pre
cision_SD  CV_Recall_Mean  CV_Recall_SD  Test_PR_AUC   Test_F1  Test_Precisi
on  Test_Recall
0  Logistic Regression                                     {'clf__C': 1.0}
0.801263      0.000823    0.702489  0.001207           0.608980         0.00
2886        0.829949      0.002055     0.803828  0.705382        0.610457
0.835264
2        Random Forest   {'clf__max_depth': 20, 'clf__n_estimators': 100}
0.738177      0.002285    0.645645  0.003798           0.554730         0.00
3609        0.772205      0.004023     0.739627  0.646018        0.546103
0.790680
1        Decision Tree  {'clf__max_depth': 10, 'clf__min_samples_split...
0.632190      0.004464    0.613016  0.000927           0.514824         0.00
1975        0.757533      0.004697     0.643501  0.621623        0.529767
0.752015

Stored best models: ['Logistic Regression', 'Decision Tree', 'Random Fores
t']
```

8. Structured-Only vs Structured + Text

This tells us whether the text column meaningfully improves beyond what structured/clinical features alone capture.

In [9]:

```
# ============================================================
# 8. STRUCTURED-ONLY VS STRUCTURED + RAW TF-IDF
#    Both pipelines are fit on df_train and evaluated on df_test
#    Uses runtime preprocessors / column lists from Cell 7
# ============================================================

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, MaxAbsScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, f1_score, precision_sco

comparison_results = []
```

```python
# ----------------------------------------------------
# Structured-only pipeline
# Rebuilt fresh from active_*_cols (Cell 7) to avoid stale-state issues
# ----------------------------------------------------
structured_only_ct = ColumnTransformer(
    transformers=[
        ("num", Pipeline([("imputer", SimpleImputer(strategy="median"))]), a
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), active_categorical_cols),
        ("binary", "passthrough", active_binary_cols),
    ],
    remainder="drop",
    sparse_threshold=1.0
)

structured_only_pipe = Pipeline([
    ("preprocess", structured_only_ct),
    ("scale", MaxAbsScaler()),
    ("clf", LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        solver="saga",
        n_jobs=-1,
        random_state=42,
        C=1.0
    ))
])

# ----------------------------------------------------
# Structured + text pipeline (uses full_preprocess_runtime from Cell 7)
# ----------------------------------------------------
structured_text_pipe = Pipeline([
    ("preprocess", full_preprocess_runtime),
    ("scale", MaxAbsScaler()),
    ("clf", LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        solver="saga",
        n_jobs=-1,
        random_state=42,
        C=1.0
    ))
])

for label, pipe in {
    "Structured only":         structured_only_pipe,
    "Structured + raw TF-IDF": structured_text_pipe
}.items():
    print("\nFitting:", label)
    pipe.fit(df_train, y_train)

    y_pred = pipe.predict(df_test)
    y_prob = pipe.predict_proba(df_test)[:, 1]
```

```
    comparison_results.append({
        "FeatureSet": label,
        "PR-AUC":     average_precision_score(y_test, y_prob),
        "F1":         f1_score(y_test, y_pred, zero_division=0),
        "Precision":  precision_score(y_test, y_pred, zero_division=0),
        "Recall":     recall_score(y_test, y_pred, zero_division=0)
    })

comparison_df = pd.DataFrame(comparison_results)
print("\nSTRUCTURED vs TEXT COMPARISON — held-out test set")
print(comparison_df.sort_values("PR-AUC", ascending=False))

comparison_df.to_csv(f"{OUTDIR}/structured_vs_raw_tfidf_comparison.csv", inc
```

```
Fitting: Structured only

Fitting: Structured + raw TF-IDF

STRUCTURED vs TEXT COMPARISON — held-out test set
                FeatureSet    PR-AUC        F1  Precision    Recall
1  Structured + raw TF-IDF  0.803828  0.705382   0.610457  0.835264
0          Structured only  0.559951  0.533096   0.431081  0.698363
```

9. Feature Importance

Extract and interpret the features driving each model's predictions:

- **Logistic Regression**: signed coefficients (positive = predicts serious; negative = predicts non-serious)
- **Random Forest**: Gini-based feature importances (unsigned, relative)
- **Decision Tree**: Gini importances + which features appear in actual tree splits + exported rule text (depth ≤ 4)

Results are saved separately for structured and text features to allow easy comparison of clinical vs. symptom-text signal.

In [10]:
```
# 9A. FIT FINAL LOGISTIC REGRESSION MODEL FOR FEATURE IMPORTANCE
#      Trained on the full training set (df_train) so coefficients
#      reflect the model that was actually evaluated on the test set.

final_logreg_pipe = Pipeline([
    ("preprocess", full_preprocess_runtime),
    ("scale", MaxAbsScaler()),
    ("clf", LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        solver="saga",
        n_jobs=-1,
        random_state=42,
        C=1.0
    ))
])
```

```
final_logreg_pipe.fit(df_train, y_train)

print("Final logistic regression model fitted on full training set.")
```

```
Final logistic regression model fitted on full training set.
```

In [11]:

```
# 9B. EXTRACT TOP FEATURES FROM LOGISTIC REGRESSION
#     (structured + raw TF-IDF)

import numpy as np
import pandas as pd

# ---------------------------------------------------
# Get fitted preprocessors
# ---------------------------------------------------
preprocessor = final_logreg_pipe.named_steps["preprocess"]
clf = final_logreg_pipe.named_steps["clf"]

# Structured feature names
structured_feature_names = preprocessor.named_transformers_["structured"].ge

# Text feature names
text_feature_names = preprocessor.named_transformers_["text"].named_steps["t

# Combine all feature names
all_feature_names = np.concatenate([structured_feature_names, text_feature_r

# Logistic regression coefficients
coefs = clf.coef_[0]

coef_df = pd.DataFrame({
    "feature": all_feature_names,
    "coefficient": coefs,
    "abs_coefficient": np.abs(coefs),
    "feature_type": ["structured"] * len(structured_feature_names) + ["text"
})

# Top positive predictors of seriousness
top_positive = coef_df.sort_values("coefficient", ascending=False).head(50)

# Top negative predictors of seriousness
top_negative = coef_df.sort_values("coefficient", ascending=True).head(50)

# Top text-only predictors
top_text_positive = coef_df[coef_df["feature_type"] == "text"].sort_values("
top_text_negative = coef_df[coef_df["feature_type"] == "text"].sort_values("

print("\nTop positive predictors of SERIOUS=1")
print(top_positive[["feature", "coefficient", "feature_type"]])

print("\nTop negative predictors of SERIOUS=1")
print(top_negative[["feature", "coefficient", "feature_type"]])

print("\nTop POSITIVE text predictors")
print(top_text_positive[["feature", "coefficient"]])
```

```
print("\nTop NEGATIVE text predictors")
print(top_text_negative[["feature", "coefficient"]])

# Save results
coef_df.to_csv(f"{OUTDIR}/logreg_all_feature_coefficients_raw_tfidf.csv", ir
top_positive.to_csv(f"{OUTDIR}/logreg_top_positive_features_raw_tfidf.csv",
top_negative.to_csv(f"{OUTDIR}/logreg_top_negative_features_raw_tfidf.csv",
top_text_positive.to_csv(f"{OUTDIR}/logreg_top_positive_text_features_raw_tf
top_text_negative.to_csv(f"{OUTDIR}/logreg_top_negative_text_features_raw_tf
```

```
Top positive predictors of SERIOUS=1
               feature  coefficient feature_type
765               care     9.700947         text
865        clinic care     5.651175         text
2458                iv     5.598860         text
4443            stroke     5.294286         text
783              cause     5.157998         text
267          admission     4.796777         text
377       appendicitis     4.683751         text
1528       epinephrine     4.599528         text
5075              went     4.369262         text
4143              sent     4.263183         text
3966          required     4.231887         text
1526               epi     4.129231         text
768       care patient     3.981522         text
3564         presented     3.820546         text
761            cardiac     3.788667         text
4808      unresponsive     3.663628         text
1529            epipen     3.653312         text
3517         pneumonia     3.650780         text
475            arrived     3.637715         text
5078         went care     3.574386         text
1555        evaluation     3.476834         text
709       breakthrough     3.283227         text
2206           hypoxia     3.249544         text
3930        remdesivir     3.248790         text
3347      patient days     3.236841         text
4413          steroids     3.209590         text
4132          seizures     3.153479         text
5063   weeks receiving     3.110530         text
4131           seizure     3.092465         text
4006         resulting     3.002237         text
2427       intravenous     2.983370         text
1722           failure     2.960818         text
4999            visits     2.943655         text
4809       unspecified     2.928231         text
3438            pepcid     2.924647         text
4150            sepsis     2.874402         text
203              acute     2.762443         text
1283         diagnosed     2.687086         text
793         cellulitis     2.644009         text
1279     dexamethasone     2.620162         text
3142     occurred date     2.617155         text
1121            damage     2.610544         text
3327           patient     2.594610         text
3692         pulmonary     2.583747         text
1514             ended     2.563417         text
772   care therapeutic     2.540741         text
3430                pe     2.519910         text
3403     patient taken     2.515907         text
1239       dehydration     2.491089         text
4187             shock     2.468566         text

Top negative predictors of SERIOUS=1
                       feature  coefficient feature_type
1305               did result    -5.977458         text
```

1588	events did	-5.190441	text
4433	stored	-3.693946	text
732	bud	-3.682791	text
3038	non onset	-3.444164	text
1658	experienced non	-3.436221	text
4910	vaccine expired	-3.379216	text
4001	result doctor	-3.365370	text
4842	use date	-3.318746	text
3897	refused	-3.204575	text
1678	expired vaccine	-3.168732	text
964	condition sender	-3.021728	text
271	adverse	-2.977986	text
862	cleared	-2.823047	text
3894	refrigerator	-2.777257	text
2392	instead	-2.758625	text
4800	unexpected non	-2.687227	text
2901	monovalent	-2.670832	text
3892	refrigerated	-2.651775	text
1933	freezer	-2.614853	text
5076	went away	-2.550683	text
625	bivalent	-2.514781	text
3035	non described	-2.491288	text
4065	routine	-2.449809	text
1619	excursion	-2.447466	text
4810	unspecified age	-2.394416	text
4878	vaccinator	-2.336264	text
2076	headache myalgia	-2.311858	text
1940	fridge	-2.300283	text
291	age	-2.300070	text
1675	expired dose	-2.273701	text
2448	itches	-2.268913	text
1089	covid treatment	-2.235182	text
4865	vaccination failure	-2.232617	text
256	administration error	-2.225090	text
4578	temperature	-2.204947	text
2256	inadvertently	-2.173662	text
5080	went home	-2.161753	text
4331	special	-2.142625	text
1385	dose dose	-2.134434	text
5040	water	-2.108830	text
3148	occurrence inappropriate	-2.100843	text
1535	error patient	-2.095555	text
3917	relevant	-2.067610	text
4428	storage	-2.065013	text
1672	expiration	-2.062063	text
4005	result vaccination	-2.050719	text
2093	health care	-2.042122	text
1471	early	-2.037881	text
5079	went doctor	-2.018890	text

Top POSITIVE text predictors

	feature	coefficient
765	care	9.700947
865	clinic care	5.651175
2458	iv	5.598860
4443	stroke	5.294286

```
783              cause     5.157998
267          admission     4.796777
377       appendicitis     4.683751
1528       epinephrine     4.599528
5075              went     4.369262
4143              sent     4.263183
3966          required     4.231887
1526               epi     4.129231
768       care patient     3.981522
3564         presented     3.820546
761            cardiac     3.788667
4808      unresponsive     3.663628
1529            epipen     3.653312
3517         pneumonia     3.650780
475            arrived     3.637715
5078         went care     3.574386
1555        evaluation     3.476834
709       breakthrough     3.283227
2206           hypoxia     3.249544
3930        remdesivir     3.248790
3347      patient days     3.236841
4413          steroids     3.209590
4132          seizures     3.153479
5063   weeks receiving     3.110530
4131           seizure     3.092465
4006         resulting     3.002237
2427       intravenous     2.983370
1722           failure     2.960818
4999            visits     2.943655
4809       unspecified     2.928231
3438            pepcid     2.924647
4150            sepsis     2.874402
203              acute     2.762443
1283         diagnosed     2.687086
793         cellulitis     2.644009
1279     dexamethasone     2.620162
3142     occurred date     2.617155
1121            damage     2.610544
3327           patient     2.594610
3692         pulmonary     2.583747
1514             ended     2.563417
772   care therapeutic     2.540741
3430                pe     2.519910
3403     patient taken     2.515907
1239       dehydration     2.491089
4187             shock     2.468566

Top NEGATIVE text predictors
                       feature  coefficient
1305                did result    -5.977458
1588                events did    -5.190441
4433                    stored    -3.693946
732                        bud    -3.682791
3038                 non onset    -3.444164
1658           experienced non    -3.436221
4910           vaccine expired    -3.379216
```

```
4001             result doctor    -3.365370
4842                  use date    -3.318746
3897                   refused    -3.204575
1678           expired vaccine    -3.168732
964           condition sender    -3.021728
271                    adverse    -2.977986
862                    cleared    -2.823047
3894              refrigerator    -2.777257
2392                   instead    -2.758625
4800            unexpected non    -2.687227
2901                monovalent    -2.670832
3892              refrigerated    -2.651775
1933                   freezer    -2.614853
5076                 went away    -2.550683
625                   bivalent    -2.514781
3035             non described    -2.491288
4065                   routine    -2.449809
1619                 excursion    -2.447466
4810           unspecified age    -2.394416
4878                vaccinator    -2.336264
2076          headache myalgia    -2.311858
1940                    fridge    -2.300283
291                        age    -2.300070
1675              expired dose    -2.273701
2448                    itches    -2.268913
1089           covid treatment    -2.235182
4865       vaccination failure    -2.232617
256       administration error    -2.225090
4578               temperature    -2.204947
2256             inadvertently    -2.173662
5080                 went home    -2.161753
4331                   special    -2.142625
1385                 dose dose    -2.134434
5040                     water    -2.108830
3148  occurrence inappropriate    -2.100843
1535             error patient    -2.095555
3917                  relevant    -2.067610
4428                   storage    -2.065013
1672                expiration    -2.062063
4005        result vaccination    -2.050719
2093               health care    -2.042122
1471                     early    -2.037881
5079               went doctor    -2.018890
```

In [12]:

```python
# ============================================================
# 9C. RANDOM FOREST FEATURE IMPORTANCE
#      Fitted on the full training set (df_train)
# ============================================================

final_rf_pipe = Pipeline([
    ("preprocess", full_preprocess_runtime),
    ("scale", MaxAbsScaler()),
    ("clf", RandomForestClassifier(
        n_estimators=100,
        max_depth=20,
        class_weight="balanced",
```

```
        random_state=42,
        n_jobs=-1
    ))
])

final_rf_pipe.fit(df_train, y_train)

preprocessor_rf = final_rf_pipe.named_steps["preprocess"]
rf = final_rf_pipe.named_steps["clf"]

structured_feature_names_rf = preprocessor_rf.named_transformers_["structure
text_feature_names_rf = preprocessor_rf.named_transformers_["text"].named_st
all_feature_names_rf = np.concatenate([structured_feature_names_rf, text_fea

rf_importance_df = pd.DataFrame({
    "feature": all_feature_names_rf,
    "importance": rf.feature_importances_,
    "feature_type": ["structured"] * len(structured_feature_names_rf) + ["te
}).sort_values("importance", ascending=False)

print("\nTop Random Forest features")
print(rf_importance_df.head(50))

rf_importance_df.to_csv(f"{OUTDIR}/random_forest_feature_importance_raw_tfic
```

```
Top Random Forest features
                                  feature  importance feature_type
2                     num__ONSET_INTERVAL    0.050410   structured
102              binary__LAB_DATA_MISSING    0.042980   structured
1                           num__NUMDAYS    0.039826   structured
203                                 acute    0.015203         text
143   binary__LAB_DATA__vitals_procedures    0.014318   structured
144               binary__LAB_DATA__other    0.013300   structured
2060                             headache    0.013017         text
0                           num__AGE_YRS    0.011242   structured
4239                                site    0.010958         text
2458                                  iv    0.010208         text
2373                           injection    0.010010         text
104       binary__HISTORY__cardiovascular    0.008309   structured
3564                           presented    0.008003         text
815                                chest    0.007916         text
417                                  arm    0.007872         text
2383                       injection site    0.007665         text
1051                               covid    0.007386         text
765                                 care    0.007360         text
1257                 describes occurrence    0.007307         text
827                                chills    0.007167         text
1373                                dose    0.006343         text
638                                blood    0.006311         text
817                            chest pain    0.006026         text
2924                                mrna    0.005954         text
5075                                went    0.005882         text
3391                     patient received    0.005751         text
715                                breath    0.005276         text
4193                           shortness    0.005071         text
4059                               route    0.005064         text
1812                               fever    0.005041         text
2887                        moderna covid    0.005004         text
92                     binary__MULTI_DOSE    0.004925   structured
1532                               error    0.004888         text
2155                                home    0.004773         text
2777                      medical history    0.004770         text
4194                     shortness breath    0.004694         text
4879                             vaccine    0.004673         text
3805                        received mrna    0.004672         text
573                                batch    0.004579         text
1256                           describes    0.004566         text
953                           concomitant    0.004339         text
225                          administered    0.004269         text
271                              adverse    0.004254         text
91                     binary__DOSE_COUNT    0.004231   structured
3144                           occurrence    0.004107         text
709                          breakthrough    0.004060         text
2773                              medical    0.003941         text
1722                              failure    0.003919         text
3517                            pneumonia    0.003881         text
1555                           evaluation    0.003804         text
```

In [13]:

```
# 9D. FIT FINAL DECISION TREE MODEL
#      Fitted on the full training set (df_train)
```

```python
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import MaxAbsScaler

final_dt_pipe = Pipeline([
    ("preprocess", full_preprocess_runtime),
    ("scale", MaxAbsScaler()),
    ("clf", DecisionTreeClassifier(
        max_depth=20,
        min_samples_split=5,
        class_weight="balanced",
        random_state=42
    ))
])

final_dt_pipe.fit(df_train, y_train)

print("Final decision tree model fitted on full training set.")
```

```
Final decision tree model fitted on full training set.
```

In [14]:

```python
# 9E. EXTRACT DECISION TREE FEATURE IMPORTANCE

# Get fitted objects
preprocessor_dt = final_dt_pipe.named_steps["preprocess"]
dt = final_dt_pipe.named_steps["clf"]

# Structured feature names
structured_feature_names_dt = preprocessor_dt.named_transformers_["structure

# Text feature names
text_feature_names_dt = preprocessor_dt.named_transformers_["text"].named_st

# Combine feature names
all_feature_names_dt = np.concatenate([structured_feature_names_dt, text_fea

# Importance values
dt_importances = dt.feature_importances_

# Put into dataframe
dt_importance_df = pd.DataFrame({
    "feature": all_feature_names_dt,
    "importance": dt_importances,
    "feature_type": ["structured"] * len(structured_feature_names_dt) + ["te
})

# Sort descending
dt_importance_df = dt_importance_df.sort_values("importance", ascending=Fals

# Top overall features
top_dt_features = dt_importance_df.head(50)

# Top text features only
top_dt_text_features = (
```

```
    dt_importance_df[dt_importance_df["feature_type"] == "text"]
    .sort_values("importance", ascending=False)
    .head(50)
)

# Top structured features only
top_dt_structured_features = (
    dt_importance_df[dt_importance_df["feature_type"] == "structured"]
    .sort_values("importance", ascending=False)
    .head(50)
)

print("\nTop overall Decision Tree features")
print(top_dt_features)

print("\nTop text Decision Tree features")
print(top_dt_text_features)

print("\nTop structured Decision Tree features")
print(top_dt_structured_features)

# Save results
dt_importance_df.to_csv(f"{OUTDIR}/decision_tree_feature_importance_raw_tfic
top_dt_features.to_csv(f"{OUTDIR}/decision_tree_top_features_raw_tfidf.csv",
top_dt_text_features.to_csv(f"{OUTDIR}/decision_tree_top_text_features_raw_t
top_dt_structured_features.to_csv(f"{OUTDIR}/decision_tree_top_structured_fe
```

Top overall Decision Tree features

	feature	importance	feature_type
2	num__ONSET_INTERVAL	0.212383	structured
102	binary__LAB_DATA_MISSING	0.168649	structured
953	concomitant	0.049833	text
144	binary__LAB_DATA__other	0.028527	structured
744	called	0.028391	text
0	num__AGE_YRS	0.019952	structured
765	care	0.019837	text
2458	iv	0.017819	text
1	num__NUMDAYS	0.017000	structured
815	chest	0.013363	text
783	cause	0.012060	text
1555	evaluation	0.010115	text
59	cat__STATE_UNKNOWN	0.009575	structured
2924	mrna	0.008629	text
3327	patient	0.008397	text
203	acute	0.008115	text
4239	site	0.007863	text
1583	events	0.007733	text
417	arm	0.007698	text
4443	stroke	0.007188	text
143	binary__LAB_DATA__vitals_procedures	0.006548	structured
5063	weeks receiving	0.005972	text
827	chills	0.005246	text
3523	positive	0.004972	text
1528	epinephrine	0.004688	text
1526	epi	0.004680	text
1051	covid	0.004296	text
4131	seizure	0.004169	text
3911	relationship	0.003904	text
715	breath	0.003896	text
5075	went	0.003855	text
1353	doctor	0.003476	text
4544	systemic	0.003475	text
784	caused	0.003454	text
104	binary__HISTORY__cardiovascular	0.003412	structured
1558	event	0.002910	text
103	binary__OTHER_MEDS_MISSING	0.002849	structured
475	arrived	0.002771	text
2373	injection	0.002757	text
2101	heart	0.002610	text
4879	vaccine	0.002546	text
3	num__MAX_DOSE	0.002471	structured
994	consumer describes	0.002329	text
3444	period	0.002307	text
84	cat__MONTH_8	0.002264	structured
2060	headache	0.002033	text
1305	did result	0.002015	text
2383	injection site	0.001991	text
4554	taken	0.001966	text
271	adverse	0.001907	text

Top text Decision Tree features

	feature	importance	feature_type
953	concomitant	0.049833	text

744	called	0.028391	text
765	care	0.019837	text
2458	iv	0.017819	text
815	chest	0.013363	text
783	cause	0.012060	text
1555	evaluation	0.010115	text
2924	mrna	0.008629	text
3327	patient	0.008397	text
203	acute	0.008115	text
4239	site	0.007863	text
1583	events	0.007733	text
417	arm	0.007698	text
4443	stroke	0.007188	text
5063	weeks receiving	0.005972	text
827	chills	0.005246	text
3523	positive	0.004972	text
1528	epinephrine	0.004688	text
1526	epi	0.004680	text
1051	covid	0.004296	text
4131	seizure	0.004169	text
3911	relationship	0.003904	text
715	breath	0.003896	text
5075	went	0.003855	text
1353	doctor	0.003476	text
4544	systemic	0.003475	text
784	caused	0.003454	text
1558	event	0.002910	text
475	arrived	0.002771	text
2373	injection	0.002757	text
2101	heart	0.002610	text
4879	vaccine	0.002546	text
994	consumer describes	0.002329	text
3444	period	0.002307	text
2060	headache	0.002033	text
1305	did result	0.002015	text
2383	injection site	0.001991	text
4554	taken	0.001966	text
271	adverse	0.001907	text
817	chest pain	0.001843	text
1193	days	0.001829	text
4634	throat	0.001815	text
2773	medical	0.001798	text
3525	positive covid	0.001791	text
2093	health care	0.001761	text
3866	red	0.001741	text
2711	lymph	0.001649	text
3065	nose	0.001585	text
4526	symptoms	0.001548	text
905	comments	0.001530	text

Top structured Decision Tree features

	feature	importance	feature_type
2	num__ONSET_INTERVAL	0.212383	structured
102	binary__LAB_DATA_MISSING	0.168649	structured
144	binary__LAB_DATA__other	0.028527	structured
0	num__AGE_YRS	0.019952	structured

```
1                                num__NUMDAYS    0.017000   structured
59                          cat__STATE_UNKNOWN    0.009575   structured
143        binary__LAB_DATA__vitals_procedures    0.006548   structured
104            binary__HISTORY__cardiovascular    0.003412   structured
103                 binary__OTHER_MEDS_MISSING    0.002849   structured
3                               num__MAX_DOSE    0.002471   structured
84                               cat__MONTH_8    0.002264   structured
69                             cat__YEAR_2021    0.001653   structured
5                                  cat__SEX_M    0.001247   structured
100                  binary__ALLERGIES_MISSING    0.001174   structured
139          binary__LAB_DATA__cardiac_markers    0.000730   structured
98                     binary__HISTORY_MISSING    0.000554   structured
101                  binary__PRIOR_VAX_MISSING    0.000473   structured
93                       binary__UNKNOWN_DOSE    0.000451   structured
91                         binary__DOSE_COUNT    0.000415   structured
105               binary__HISTORY__respiratory    0.000402   structured
71                             cat__YEAR_2023    0.000381   structured
83                               cat__MONTH_7    0.000359   structured
33                              cat__STATE_MI    0.000352   structured
78                               cat__MONTH_2    0.000350   structured
151                 binary__OTHER_MEDS__other    0.000323   structured
75                              cat__MONTH_10    0.000291   structured
48                              cat__STATE_OK    0.000260   structured
82                               cat__MONTH_6    0.000245   structured
79                               cat__MONTH_3    0.000238   structured
99                     binary__CUR_ILL_MISSING    0.000238   structured
70                             cat__YEAR_2022    0.000226   structured
129            binary__ALLERGIES__drug_allergy    0.000221   structured
128                     binary__CUR_ILL__other    0.000215   structured
61                              cat__STATE_VA    0.000206   structured
4                                  cat__SEX_F    0.000200   structured
25                              cat__STATE_IN    0.000189   structured
34                              cat__STATE_MN    0.000176   structured
89                binary__MANU__PFIZER_BIONTECH    0.000157   structured
43                              cat__STATE_NJ    0.000155   structured
29                              cat__STATE_MA    0.000152   structured
111                    binary__HISTORY__kidney    0.000145   structured
37                              cat__STATE_MS    0.000144   structured
94                  binary__MULTI_MANUFACTURER    0.000144   structured
146                 binary__OTHER_MEDS__statin    0.000136   structured
133                   binary__ALLERGIES__other    0.000136   structured
115                binary__HISTORY__neurologic    0.000134   structured
87                       binary__MANU__MODERNA    0.000133   structured
114  binary__HISTORY__cancer_immunocompromised    0.000129   structured
74                               cat__MONTH_1    0.000126   structured
131     binary__ALLERGIES__latex_other_contact    0.000121   structured
```

In [15]:

```
# 9F. FEATURES USED IN DECISION TREE SPLITS

used_feature_idx = dt.tree_.feature
used_feature_idx = used_feature_idx[used_feature_idx >= 0]   # remove leaf n

used_feature_names = all_feature_names_dt[used_feature_idx]
used_feature_names_unique = pd.Series(used_feature_names).value_counts().res
used_feature_names_unique.columns = ["feature", "num_splits"]
```

```python
print("\nFeatures actually used in tree splits")
print(used_feature_names_unique.head(50))

used_feature_names_unique.to_csv(
    f"{OUTDIR}/decision_tree_features_used_in_splits_raw_tfidf.csv",
    index=False
)
```

```
Features actually used in tree splits
                      feature  num_splits
0                 num__AGE_YRS          66
1                 num__NUMDAYS          31
2                      patient          28
3          num__ONSET_INTERVAL          22
4                        covid          20
5                      vaccine          18
6                          arm          18
7                         care          18
8                         days          16
9                         went          15
10                    positive          15
11                   injection          14
12                        pain          13
13                      lasted          12
14                         did          12
15  binary__OTHER_MEDS_MISSING          12
16                       fever          12
17                        site          12
18                         day          12
19                        rash          12
20                      second          12
21                    headache          11
22                     started          11
23                     fatigue          11
24                      chills          11
25                       heart          11
26                    symptoms          10
27                        dose          10
28               num__MAX_DOSE          10
29                       chest          10
30                     medical          10
31                      doctor          10
32                        just           9
33               positive covid           9
34                       weeks           9
35                         red           9
36            patient received           8
37                       blood           8
38                          th           8
39                        shot           8
40                        body           8
41                 vaccination           8
42                    received           8
43                    consumer           8
44   binary__ALLERGIES_MISSING           8
45                     moderna           7
46                        date           7
47                     booster           7
48                      better           7
49                      throat           7
```

In [16]:

```python
# 9G. EXPORT DECISION TREE RULES

from sklearn.tree import export_text
```

```python
tree_rules = export_text(
    dt,
    feature_names=list(all_feature_names_dt),
    max_depth=4
)

print(tree_rules)

with open(f"{OUTDIR}/decision_tree_rules_raw_tfidf.txt", "w") as f:
    f.write(tree_rules)
```

```
|--- num__ONSET_INTERVAL <= 0.00
|   |--- binary__LAB_DATA_MISSING <= 0.50
|   |   |--- binary__LAB_DATA__other <= 0.50
|   |   |   |--- mrna moderna <= 0.37
|   |   |   |   |--- available appropriate <= 0.39
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- available appropriate >  0.39
|   |   |   |   |   |--- truncated branch of depth 6
|   |   |   |--- mrna moderna >  0.37
|   |   |   |   |--- relationship <= 0.12
|   |   |   |   |   |--- truncated branch of depth 4
|   |   |   |   |--- relationship >  0.12
|   |   |   |   |   |--- truncated branch of depth 2
|   |   |--- binary__LAB_DATA__other >  0.50
|   |   |   |--- concomitant <= 0.21
|   |   |   |   |--- arm <= 0.05
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- arm >  0.05
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- concomitant >  0.21
|   |   |   |   |--- care <= 0.03
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- care >  0.03
|   |   |   |   |   |--- truncated branch of depth 15
|   |--- binary__LAB_DATA_MISSING >  0.50
|   |   |--- called <= 0.08
|   |   |   |--- iv <= 0.04
|   |   |   |   |--- cause <= 0.06
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- cause >  0.06
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- iv >  0.04
|   |   |   |   |--- events did <= 0.07
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- events did >  0.07
|   |   |   |   |   |--- truncated branch of depth 2
|   |   |--- called >  0.08
|   |   |   |--- arrived <= 0.03
|   |   |   |   |--- taken <= 0.17
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- taken >  0.17
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- arrived >  0.03
|   |   |   |   |--- home <= 0.08
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- home >  0.08
|   |   |   |   |   |--- truncated branch of depth 4
|--- num__ONSET_INTERVAL >  0.00
|   |--- concomitant <= 0.15
|   |   |--- num__NUMDAYS <= 0.00
|   |   |   |--- site <= 0.11
|   |   |   |   |--- binary__LAB_DATA__vitals_procedures <= 0.50
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- binary__LAB_DATA__vitals_procedures >  0.50
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- site >  0.11
```

```
|   |   |   |   |--- red <= 0.20
|   |   |   |   |   |--- truncated branch of depth 15
|   |   |   |   |--- red >  0.20
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |--- num__NUMDAYS >  0.00
|   |   |   |--- num__AGE_YRS <= 0.54
|   |   |   |   |--- positive <= 0.21
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- positive >  0.21
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- num__AGE_YRS >  0.54
|   |   |   |   |--- doctor <= 0.14
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- doctor >  0.14
|   |   |   |   |   |--- truncated branch of depth 16
|   |--- concomitant >  0.15
|   |   |--- events <= 0.04
|   |   |   |--- event <= 0.05
|   |   |   |   |--- binary__ALLERGIES_MISSING <= 0.50
|   |   |   |   |   |--- truncated branch of depth 4
|   |   |   |   |--- binary__ALLERGIES_MISSING >  0.50
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- event >  0.05
|   |   |   |   |--- dose mrna <= 0.20
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- dose mrna >  0.20
|   |   |   |   |   |--- truncated branch of depth 7
|   |   |--- events >  0.04
|   |   |   |--- consumer describes <= 0.28
|   |   |   |   |--- binary__LAB_DATA__vitals_procedures <= 0.50
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |   |--- binary__LAB_DATA__vitals_procedures >  0.50
|   |   |   |   |   |--- truncated branch of depth 16
|   |   |   |--- consumer describes >  0.28
|   |   |   |   |--- events <= 0.14
|   |   |   |   |   |--- truncated branch of depth 9
|   |   |   |   |--- events >  0.14
|   |   |   |   |   |--- truncated branch of depth 13
```